1D-RS vs 2D-RS: how they work, and how they compare

> **History.** This document was first written when `2D_RS_Galois.py` extended each axis 64→128, giving a 128×128 codeword at 4× overhead and rate 0.25 — which made any comparison against the 2× schemes unfair. The geometry moved to 88×88 (1.89×) and then to the current **88×96 at 2.06× overhead and rate 0.485**, the closest any grid reachable in this field gets to rate 0.5. The text and numbers below describe the current construction, and the results come from the shared benchmark (`benchmark_results.json`). The natural-looking 90×90 cannot be built: a length-n NTT needs n to divide $p-1$, and $90 = 2 \cdot 3^2 \cdot 5$ needs a 3^2 where this field has only 3^1, plus a 5 it does not have at all.

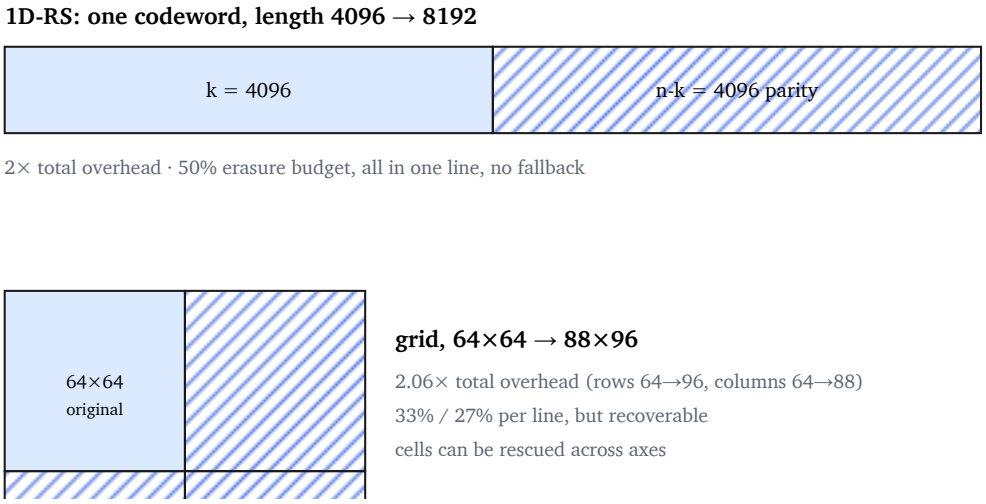
This document explains the two coding schemes conceptually and summarises what running the simulations actually shows, framed against the report's methodology (Phase 2 analytical modelling / Phase 3 empirical benchmarking) so it can be adapted directly into the write-up. For implementation detail and the underlying maths, see 1D_2D_RS_code_walkthrough.md.

Both simulations are built on the same foundation on purpose: identical BLS12-381 scalar field, identical EIP-4844 blob-packing convention (via `blob_generator.py`), the same bit-reversal domain ordering wherever a length allows it (the 88- and 96-length 2D axes fall back to natural NTT order, since bit reversal is defined only for powers of two — an ordering convention, not a change to the maths), identical FFT-based erasure-decode machinery. The only thing that differs between the two files is the coding scheme's *structure* — one axis vs two. That's what makes the comparison below a fair one rather than two unrelated implementations measuring different things.

Background: why "evaluation form" matters here

Ethereum's blob commitment scheme (KZG, per EIP-4844) requires a blob's 4096 field elements to be interpreted as the **evaluations** of a polynomial at the 4096th roots of unity, not as its coefficients — this is what lets a KZG commitment be computed directly against a Lagrange-basis trusted setup with no transform needed at commit time. Reed-Solomon extension of a blob is therefore fundamentally an *interpolate, then evaluate on a bigger domain* operation: recover the low-degree polynomial's coefficients from the known evaluations (inverse NTT), then evaluate that same polynomial at more points (NTT). Both schemes below extend this way, which is what keeps the comparison compatible with "maintaining the use of KZG commitments" as the report's stated design constraint. (The walkthrough document works this whole pipeline through with real numbers on a toy field, if you want to see every intermediate value.)

The structural difference, at a glance



1D-RS extends one long codeword to double its length (2× overhead, 50% erasure ceiling with no fallback if exceeded). 2D-RS extends a 64×64 grid along both axes, to 96 columns and 88 rows (2.06× overhead; a row survives 32 of 96 erasures and a column 24 of 88, but individual cells can be rescued across axes). Schematic: the blocks are not drawn to scale.

1D-RS extends one long codeword; 2D-RS extends a grid along two independent axes. Every other difference discussed below — erasure-tolerance behaviour, decode cost, guarantees — is a *consequence* of this one structural choice, not an independent design decision.

1D-RS: the PeerDAS baseline today

The blob is treated as **one** codeword. All 4096 evaluations define a single degree-`<4096` polynomial; extension evaluates that polynomial at 8192 points instead of 4096 — a clean **2× total overhead**, which is exactly what PeerDAS uses today ([10], [11]).

Recovery is a single erasure-decode pass over the whole 8192-length codeword. Reed-Solomon's textbook guarantee applies directly and unforgivingly: if at most `n − k = 4096` (50%) of the codeword is missing, the polynomial (and therefore every missing sample) is recoverable exactly. If more than that is missing, there is nothing else to try — no partial credit, no second attempt from a different angle. The simulation's decoder reflects this precisely: it's a single function call that returns either a fully recovered codeword or an explicit, immediate failure.

This gives 1D-RS a simple, easy-to-reason-about erasure-tolerance model (exactly 50%, no exceptions) and a decode cost dominated by one interpolate step and — when erasures are present — one polynomial division against a divisor whose degree equals the erasure count.

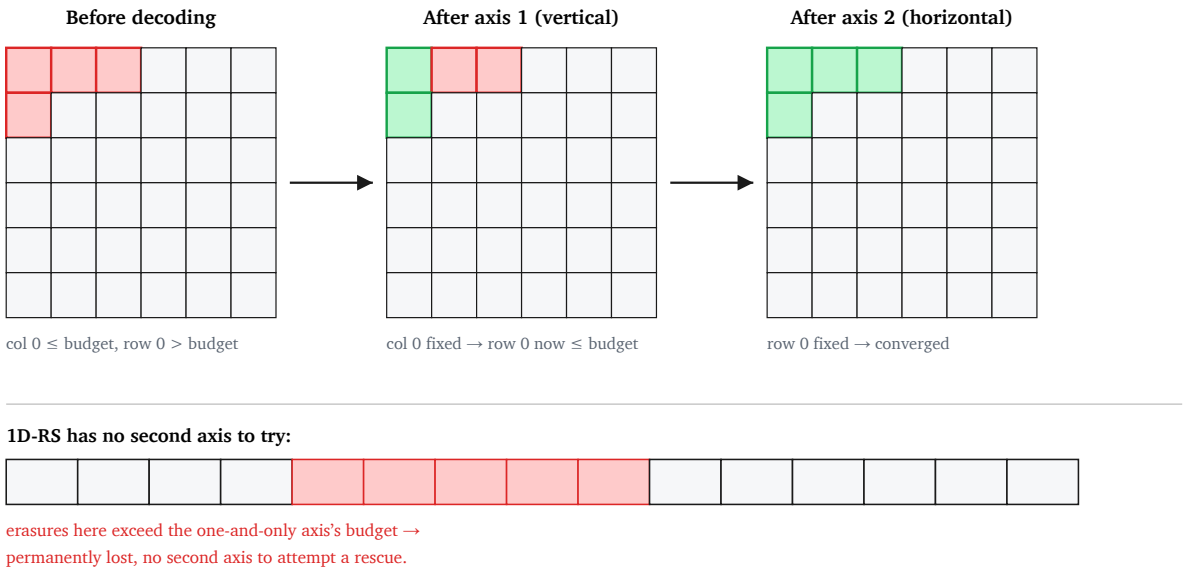
2D-RS: the product code

The blob is reshaped into a 64×64 grid. Every row is extended from 64 to 96 values and every column of the result from 64 to 88, giving an **88×96 codeword of 8448 symbols — 2.06× overhead, rate 0.485**. The component codes are [96, 64] along a row and [88, 64] down a column. Exactly 2× is unreachable for a genuine product code on a 4096-element blob in this field, and 88×96 is the closest reachable pair; the 3% difference is reported rather than hidden, and it runs against 2D-RS.

Structurally, the 64×64 array of original values defines a bivariate polynomial `P(x,y) = Σ c_{i,j} x^i y^j` of degree `<64` in each variable independently. Fixing either variable at *any* value — including one of the extension points that variable's axis introduces, not just an original point — leaves a polynomial in the other variable that is still degree `<64` (holding one variable fixed can only collapse coefficients together, never raise the degree in the variable left free). Because of that, **every** row and **every** column of the fully-extended 88×96 grid — not just the original 64 — is itself a valid Reed-Solomon codeword. That single fact is what makes 2D-RS a genuine product code rather than "two unrelated 1D-RS codes stapled together", and it's what the decoder relies on. (The walkthrough document derives this in full; its worked examples use the original 64→128 geometry, but the argument is the same.)

Where 2D-RS's advantage over 1D-RS actually comes from

Per line, 2D-RS's erasure budget is far smaller than 1D-RS's: a row of 96 can lose at most 32 values (33%) and a column of 88 at most 24 (27%), against 1D-RS's 4096 of 8192 (50%). Taken alone, that is worse, not better. What makes up the difference is **iteration between the two axes**: a cell whose row and column each have too many erasures to recover alone can still become recoverable once *either* axis clears enough of its neighbours for the *other* to fall within budget.



The same rescue mechanism as the walkthrough document's worked example, with a direct 1D-RS contrast: the strip at the bottom has no second axis to fall back on, so erasures beyond its one budget are simply unrecoverable — the structural reason a product code tolerates more than either axis's own budget would suggest in isolation.

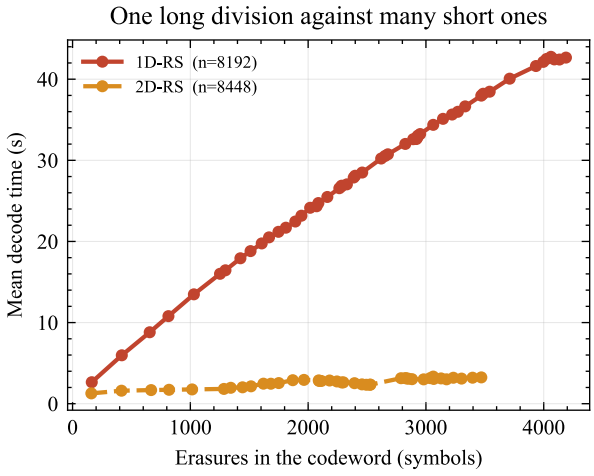
This is genuinely iterative — the decoder alternates vertical and horizontal passes until convergence, not a fixed two-pass schedule — and at this geometry it does real work even on randomly drawn patterns. Measured on five random masks at each rate: at 15% erasure no line exceeds its budget and every decode finishes in one round; at 35%, 59% of rows and 92% of columns are over their own budget, yet every decode converges in two rounds; at 38%, with 80% of rows and 98% of columns over budget, in three. 1D-RS, with no second axis, has no equivalent recourse: as the strip at the bottom of the figure shows, erasures that exceed the one axis's budget are simply gone. This isn't a missing feature of the 1D-RS script; it's a structural fact about having a single axis, and it's the mechanism-level reason a product code is more than "the same code, run twice."

At matched storage the comparison is direct. 1D-RS is MDS: it survives any pattern with at most half the codeword missing, and under random erasure its 50% threshold measures 0.499, against an exact 0.50006. 2D-RS reaches 0.400 — less than 1D-RS, despite storing 3% more. Its *guaranteed* tolerance is far lower still: a product code's minimum distance is the product of its component distances, $33 \times 25 = 825$, so it is guaranteed to survive only $(d-1)/n = 0.098$ of the codeword against a worst-case pattern; its 0.400 on random patterns is four times that. What 2D-RS buys with the tolerance it gives up is decode speed.

Results from the shared benchmark

The numbers below come from the shared erasure sweep in `benchmark_1D_2D_BC.py`: both schemes encode the same blob at the same erasure rates, and a trial counts only if the recovered bytes match exactly. Full detail, and the five block-circulant rows alongside, is in benchmark_1D_2D_BC_report.md.

	1D-RS	2D-RS
Codeword	8192 symbols, one line	88×96 = 8448 symbols
Overhead / rate	2.00× / 0.500	2.06× / 0.485
Erasure budget per line	4096 of 8192 (50%), single shot	32 of 96 per row (33%), 24 of 88 per column (27%), iterable across axes
Worst-case guarantee, $(d-1)/n$	0.500 (MDS)	0.098 ($d = 825$)
50% threshold, random erasure	0.499 (95% CI 0.497–0.501)	0.400 (95% CI 0.397–0.403)
Recovers every trial up to / none from	49% / 51.5%	38% / 42%
Decode at 2% / 10% / 20% / 33% erasure	2.65 / 10.79 / 20.50 / 30.22 s	1.27 / 1.72 / 2.46 / 3.12 s
Silent corruption	0 of 132 trials	0 of 272 trials



Measured decode time vs. total erasures in the codeword, regenerated by `make_figures.py` from the sweep in `benchmark_results.json`. Plotted against erasure *count* rather than rate because the two codes differ in length (8192 against 8448), so equal rates are not equal repair work. 1D-RS's single polynomial division scales with the total erasure count directly; 2D-RS's many small per-line divisions barely move even at higher total erasure counts.

Two things are worth calling out explicitly, because they're genuine findings rather than implementation artefacts:

1. **Decode cost follows the structure, not the erasure rate.** 1D-RS's decode time climbs with every erasure — 2.65 s at 163 erasures, 10.79 s at 814, 20.50 s at 1668, about 12 ms per erasure — because every erasure adds to the degree of the *one* polynomial being divided, and the `galois` library's division is schoolbook-style. 2D-RS's barely moves — 1.27 s at 160 erasures, 2.46 s at 1684, 3.12 s at 2835 — because its erasures are spread across up to 184 short lines (88 rows and 96 columns), each dividing a divisor of degree at most 32. At 20% erasure 2D-RS does the repair about eight times faster. That asymmetry is a real computational-complexity datapoint: "one large codeword" against "many small codewords" is exactly the structural trade-off the comparison is meant to surface.
2. **At this geometry the cross-axis cascade matters on random patterns too.** With per-line budgets of 33% and 27%, random erasure at 35% puts most lines over budget, so the decoder can only succeed by iterating — which it does, up to the 0.400 threshold. In the original 128×128 construction, with 50% per line, random patterns at 35% stayed inside every line's budget and decoded in a single round; that is no longer the case. A structure-aware adversary remains the sharper threat: the 0.098 guarantee is what a targeted pattern can force, far below the random-pattern 0.400 — a gap 1D-RS, being MDS, does not have.

What this evidence does and doesn't support

Supports directly:

- At matched storage (2.06× against 2×), 1D-RS tolerates more random erasure (0.499 against 0.400) and is guaranteed to tolerate far more adversarial erasure (0.500 against 0.098), while 2D-RS decodes several times faster (about eight times at 20%).
- 2D-RS's tolerance is a property of *how erasures are distributed*, not just *how many*: the cascade earns 0.400 on random patterns, while a targeted pattern of 825 erasures — under 10% of the codeword — can defeat it.
- Both schemes reconstruct to the exact original bytes end to end, through the full EIP-4844 packing/unpacking round trip, with silent corruption at zero across every trial — the baseline correctness bar that timing numbers depend on.

Not covered by these two scripts:

- KZG proof generation/verification cost — the scripts model the erasure code, not the commitment scheme sitting on top of it, so no claim is made about the "maintaining KZG commitments" constraint beyond "the evaluation-form data model stays compatible with it".
- Network behaviour, which is measured separately by `network_sim.py` reusing the same adapters — see network_sim_report.md.

Since this was first written

- `BCcodeEthereum.py` reached blob scale on the real field with byte-level round-trip verification, so block-circulant codes joined the comparison on equal footing.
- `benchmark_1D_2D_BC.py` now runs all seven schemes over the same erasure sweep, with the same blob and, where codeword lengths match, the same erasure positions — the controlled sweep this document originally called for.
- `network_sim.py` adds the network layer: realised erasure under peer withholding, gather time, bandwidth and per-slot throughput.